MA 5755: Data Analysis & Visualization in R/Python/SQL

Week 6: Principal Components Analysis

Dr. Rakhi Singh
Department of Mathematics
IIT Madras

Spring 2026

Why Principal Component Analysis (PCA) now?

So far we have studied clustering methods that attempt to identify groups of observations in high–dimensional datasets.

Challenge in high dimensions.

- Modern datasets often contain a large number of correlated variables.
- Understanding structure using pairwise plots becomes infeasible.
- Number of pairwise scatterplots for p variables:

$$\frac{p(p-1)}{2}$$

- Example:
  - $p = 10 \Rightarrow 45$ plots
  - $p = 20 \Rightarrow 190$ plots

Instead of studying the original variables directly, we seek a small set of **derived variables** that capture most of the variability in the data.

Principal Component Analysis:overview

PCA: Find a new coordinate system consisting of directions along which the data vary the most.

- A linear transformation of the data.
- Produces new orthogonal variables called **principal components**.
- Ordered so that each component explains the maximal remaining variance.

Its role?

- Dimensionality reduction
- Visualization of high–dimensional data
- Preprocessing for clustering and nonlinear embeddings (e.g., t-SNE, UMAP)

Principal Component Analysis (PCA)

When a dataset contains many correlated variables, PCA constructs a smaller set of variables that summarize most of the variability.

Let the data matrix be $X \in \mathbb{R}^{n \times p}$, with n observations and p variables. Each observation lies in a p–dimensional space.

Idea. Find a new coordinate system consisting of directions along which the data vary the most. These directions define the **principal components**.

- First PC: direction of maximum variance
- Second PC: orthogonal direction with the next highest variance
- Continue sequentially

Thus PCA finds a **low–dimensional representation** of the data while preserving as much variability as possible.

Data Setup and Notation

Let the dataset consist of n observations of p variables.

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{bmatrix} \in \mathbb{R}^{n \times p}, \qquad x_i \in \mathbb{R}^p$$

Each row x_i is a p–dimensional observation.

Population mean

$$\mu = E[X] \in \mathbb{R}^p$$

Population covariance matrix

$$\Sigma = E[(X - \mu)(X - \mu)^T] \in \mathbb{R}^{p \times p}$$

In practice, μ and Σ are unknown and must be estimated from the data.

Sample Mean and Sample Covariance

Given observations $\{x_1, \ldots, x_n\}$:

Sample mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

Centered data

$$x_i^c = x_i - \bar{x}$$

Sample covariance matrix

$$S = \frac{1}{n-1} X_c^T X_c$$

- $S \in \mathbb{R}^{p \times p}$
- S_{jk} measures covariance between variables j and k

Goal of Principal Component Analysis

Each observation x_i lies in $\mathbb{R}^p$. When p is large, the data may lie near a lower–dimensional subspace.

Objective: Find orthogonal directions

$$\phi_1, \phi_2, \ldots, \phi_p$$

such that projections of the data onto these directions capture maximal variance.

The projections are called **principal components**.

$$z_i = \phi^T x$$

The components are ordered so that

$$Var(z_1) \geq Var(z_2) \geq \cdots \geq Var(z_p)$$

First Principal Component

Consider a unit vector $\phi \in \mathbb{R}^p$, $\qquad \|\phi\| = 1$.

Project a centered observation x onto ϕ, that is, $z = \phi^T x$.

Variance of the projection:

$$Var(z) = Var(\phi^T x)$$

Using covariance properties,

$$Var(\phi^T x) = \phi^T S \phi$$

Optimization problem

$$\phi_1 = \arg \max_{\|\phi\|=1} \phi^T S \phi$$

Solving the Optimization Problem

Maximize $\phi^T S \phi$ subject to $\phi^T \phi = 1$.

Use a Lagrange multiplier:

$$\mathcal{L}(\phi, \lambda) = \phi^T S \phi - \lambda(\phi^T \phi - 1)$$

Take derivative w.r.t. ϕ:

$$\frac{\partial \mathcal{L}}{\partial \phi} = 2S\phi - 2\lambda\phi = 0$$

which gives

$$S\phi = \lambda\phi$$

Thus ϕ must be an eigenvector of the covariance matrix S.

Connection to Eigenvalue Decomposition

Let $S = P\Lambda P^T$ be the eigendecomposition of the covariance matrix.

- $P = [\phi_1, \ldots, \phi_p]$ contains eigenvectors
- $\Lambda = diag(\lambda_1, \ldots, \lambda_p)$ contains eigenvalues

with

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$$

Result:

- First principal component direction = eigenvector with largest eigenvalue
- Variance explained by $\text{PC}_i = \lambda_i$

$$Var(z_i) = \lambda_i$$

Principal Component Scores

Let $P = [\phi_1, \phi_2, \ldots, \phi_p]$ be the eigenvector matrix.

The principal component scores are obtained by p**rojecting the centered data** $Z = X_c P$, where

$$Z \in \mathbb{R}^{n \times p}$$

Interpretation:

- rows of Z = observations in PC coordinates
- columns of Z = principal components

Thus PCA represents the data in a new orthogonal coordinate system.

Note that the components are uncorrelated, that is,
$Cov(Z_i, Z_j) = 0, \quad i \neq j$

Example

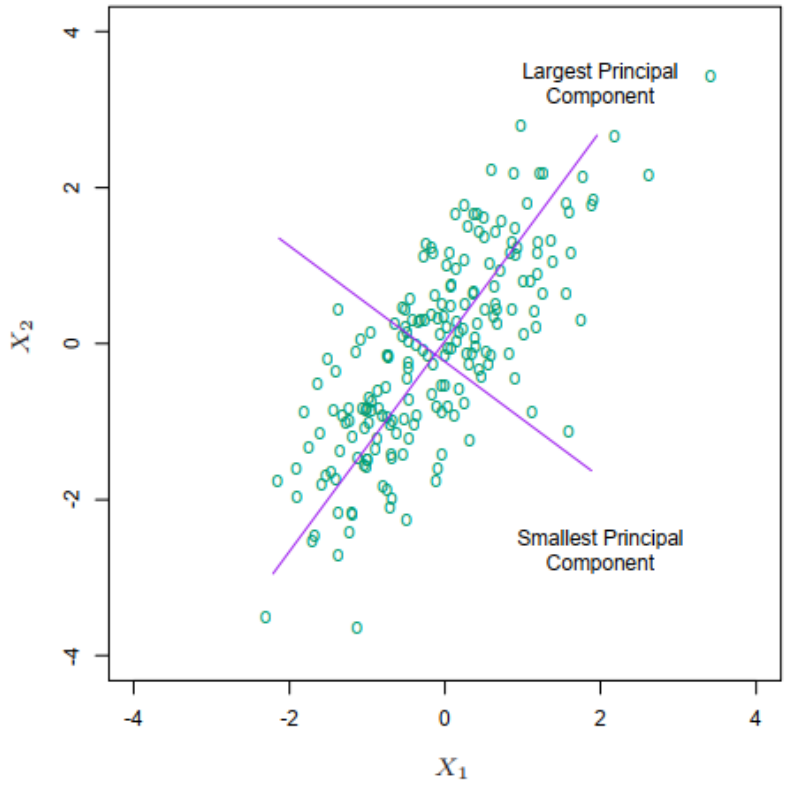


FIGURE 3.9. *Principal components of some input data points. The largest principal component is the direction that maximizes the variance of the projected data, and the smallest principal component minimizes that variance. Ridge regression projects* $\mathbf{y}$ *onto these components, and then shrinks the coefficients of the low–variance components more than the high-variance components.*

Principal Components, Scores, and Loadings

Principal Components

The k-th principal component is

$$PC_k = X_c \phi_k$$

Scores

Scores are the coordinates of observations in the PC space:

$$z_{ik} = x_i^T \phi_k$$

They represent where observation i lies along component k.

Loadings

Loadings are the coefficients of the eigenvectors ϕ_k. They indicate how strongly each original variable contributes to a component.

Example

Suppose we have two variables: $x_1 = \text{Math}$, $x_2 = \text{Physics}$. Assume PCA gives eigenvector $v_1 = \begin{bmatrix} 0.71 \\ 0.71 \end{bmatrix}$.

Principal Component:

$$PC_1 = 0.71x_1 + 0.71x_2$$

The numbers 0.71 are the **loadings**.

Score:

If a student has centered values $x = (10, 12)$, then the PC score is

$$z_1 = 10(0.71) + 12(0.71) = 15.62$$

Thus:

- Loadings define the component, Principal component is the linear combination, and Scores are the projected values for observations.

Variance Explained

Total variance in the data:

$$\mathrm{tr}(S) = \sum_{j=1}^{p} \lambda_j$$

Variance explained by the first k principal components:

$$\sum_{i=1}^{k} \lambda_i$$

Proportion of variance explained

$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{p} \lambda_j}$$

This criterion is used to decide how many components to retain.

Practical Considerations in PCA

Principal components are not scale invariant.

If variables are measured in different units:

- Variables with large variance dominate the PCs
- Interpretation becomes misleading

Standard practice
Before applying PCA:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

- **Center** each variable and **Scale** to unit variance

Thus, PCA is typically applied to the **correlation matrix** rather than the covariance matrix when variables have different scales.

PCA via Singular Value Decomposition

Instead of computing eigenvectors of S, numerical libraries use the SVD of the centered data matrix.

$$X_c = U\Sigma V^T$$

- U : orthogonal matrix of left singular vectors
- Σ : diagonal matrix of singular values
- V : orthogonal matrix of right singular vectors

Substituting into $S = \frac{1}{n-1} X_c^T X_c$ gives $S = \frac{1}{n-1} V\Sigma^2 V^T$.

Therefore,

- principal directions = columns of V, and eigenvalues = $\Sigma^2/(n-1)$

Most PCA implementations (NumPy, scikit-learn) compute PCA using SVD for numerical stability.

MNIST Example

Consider a collection of handwritten images of the digit **3** from the MNIST dataset.

Applying PCA gives principal component directions $v_1, v_2, \ldots$ that describe the main modes of variation among the images.

$\approx$ 50 PCs explain about 90% of the variation.

$$\begin{aligned} \hat{f}(\lambda) &= \bar{x} + \lambda_1 v_1 + \lambda_2 v_2 \\ &= \boxed{} + \lambda_1 \cdot \boxed{} + \lambda_2 \cdot \boxed{}. \end{aligned}$$

- v_1**:** controls the *shape of the lower tail* of the digit "3". Moving along $PC1$ lengthens or shortens the lower curve.
- v_2**:** controls *character thickness*. Increasing $PC2$ makes the strokes thicker.

Motivation: Limitations of Linear PCA

Principal Component Analysis assumes that the data structure is linear.

However, many datasets lie on nonlinear manifolds embedded in a high-dimensional space.

Example:

- concentric circles
- spiral structures
- curved manifolds

In such cases:

- linear projections cannot separate the structure
- PCA fails to capture the intrinsic geometry

Idea: map the data into a higher-dimensional feature space where linear PCA becomes effective.

Feature Space Transformation

Let

$$x_i \in \mathbb{R}^p$$

be the original data.

Define a nonlinear mapping

$$\phi : \mathbb{R}^p \to \mathcal{H}$$

into a high-dimensional feature space.

PCA is then performed on transformed data

$$\phi(x_1), \phi(x_2), \ldots, \phi(x_n)$$

Problem:

- explicit computation of $\phi(x)$ may be expensive or infinite-dimensional

Kernel Trick

Instead of computing $\phi(x)$ explicitly, we compute inner products in feature space.

$$k(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

The function $k(\cdot, \cdot)$ is called a **kernel function**.

Define the kernel matrix

$$K_{ij} = k(x_i, x_j)$$

Thus all computations can be expressed in terms of the kernel matrix K without explicitly computing $\phi(x)$.

Kernel PCA Formulation

Let

$$K \in \mathbb{R}^{n \times n}$$

be the kernel matrix.

After centering the kernel matrix, solve

$$K\alpha = \lambda\alpha$$

where

- α are eigenvectors
- λ are eigenvalues

The projection of observation x onto the k-th component is

$$z_k(x) = \sum_{i=1}^{n} \alpha_i^{(k)} k(x_i, x)$$

Kernel PCA Algorithm

Step 1
Compute kernel matrix

$$K_{ij} = k(x_i, x_j)$$

Step 2
Center the kernel matrix.

Step 3
Solve the eigenvalue problem

$$K\alpha = \lambda\alpha$$

Step 4
Use eigenvectors to compute projections in the principal component space.

Common Kernel Functions

Polynomial kernel

$$k(x_i, x_j) = (x_i^T x_j + c)^d$$

Gaussian (RBF) kernel

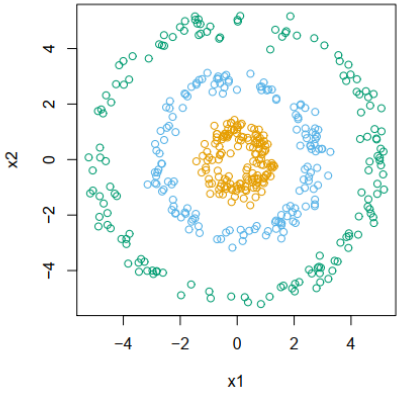
$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

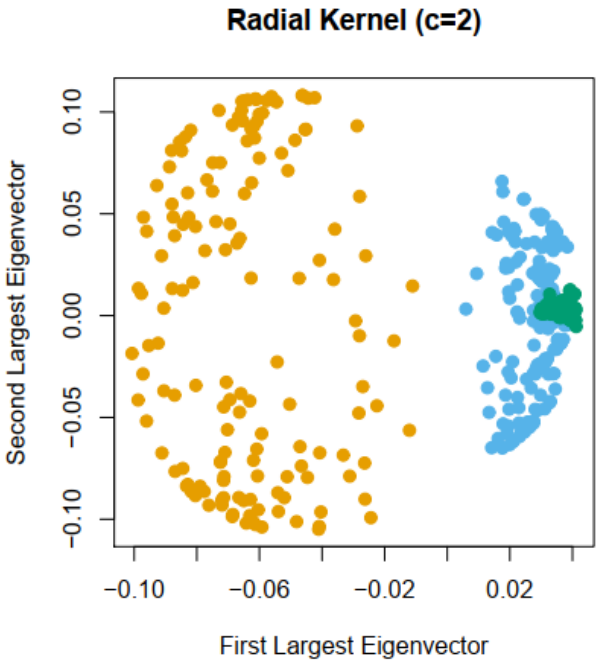
Sigmoid kernel

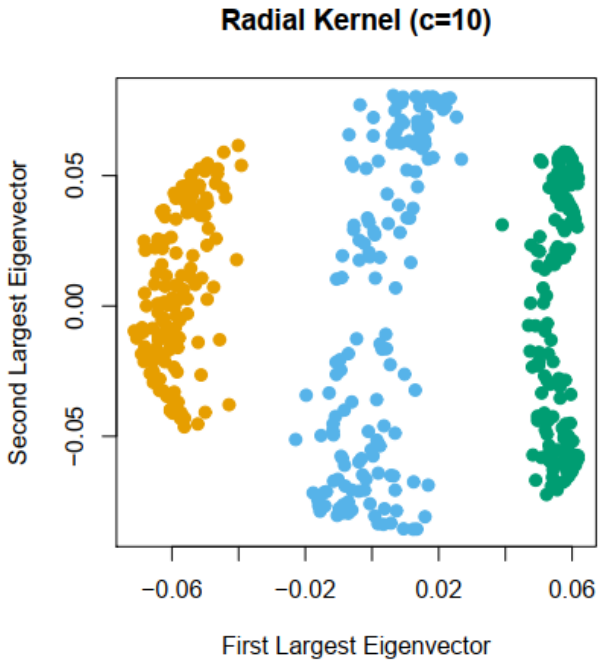
$$k(x_i, x_j) = \tanh(\alpha x_i^T x_j + c)$$

The kernel determines the geometry of the feature space.

Example


x1
x2
−4
−2
0
2
4


Radial Kernel (c=2)
Second Largest Eigenvector
First Largest Eigenvector
−0.10
−0.05
0.00
0.05
0.10
−0.10
−0.06
−0.02
0.02


Radial Kernel (c=10)
Second Largest Eigenvector
First Largest Eigenvector
−0.05
0.00
0.05
−0.06
−0.02
0.02
0.06

Summary

Kernel PCA extends PCA to nonlinear structures.

Key ideas:

- map data to a high-dimensional feature space
- use kernel functions to avoid explicit computation
- perform PCA using the kernel matrix

Applications:

- nonlinear dimensionality reduction
- image analysis
- pattern recognition